Let's go ahead and stop the web service:

```
$ docker-compose stop web
Stopping myapp_web_1 ... done
```

Loading localhost:3000 in the browser will now fail, and listing the containers will report that the Rails server has terminated:

```
$ docker-compose ps
   Name              Command           State    Ports
-------------------------------------------------------
myapp_web_1   bin/rails s -b 0.0.0.0   Exit 1
```

Having stopped a container, we can start it again with the start command:

```
$ docker-compose start web
Starting web ... done
```

There's also a restart command that's handy if, for example, you've made some config changes and want the Rails server to pick them up.

```
$ docker-compose restart web
Restarting myapp_web_1 ... done
```

The various Compose commands we've seen all rely on underlying docker commands.[3] However, we won't cover those in detail since we'll be using Compose from now on. Compose gives us all the power of the lower-level docker commands, but with simpler, app-centric commands.

Other Common Tasks

Besides starting and stopping our services, there are a few more things that we do frequently as part of our day-to-day development. We're going to take a whistle-stop tour of the highlights, since we'll be needing them in the upcoming chapters.

Viewing the Container Logs

We've seen that the docker-compose up, without the -d option, attaches to the containers it starts and follows their output. However, there's also a dedicated command for viewing the container output, which is more flexible.

Let's view the container logs:

```
$ docker-compose logs -f web
```

3. https://docs.docker.com/engine/reference/commandline/start/